

Multimodal Fashion & Context Retrieval

Glance ML Internship Assignment — Design Write-up Codebase: <https://github.com/prakhar-raj-031104/FASHIONATTIRE>

0. Executive summary

I built a natural-language image search engine over **3,200 fashion images** that reasons jointly about **what** a person wears, **where** they are, and the **vibe** of the outfit.

The core thesis: *a single global CLIP embedding is structurally incapable of compositional retrieval, so the fix must add structure — not a bigger encoder.* The system therefore pairs a **fashion-domain encoder (FashionCLIP)** with three complementary signals, the most important being **per-garment region decomposition**, which binds each colour to the specific garment it modifies.

Corpus	3,200 images · 9,851 garment regions · FAISS (3 indexes) + SQLite
Latency	~150 ms per query warm (318 ms mean incl. cold start), 2-stage ANN recall → rerank
Key result	On a 12-pair colour-swap test, vanilla CLIP returns 75.0% identical results for a query and its colour-swapped twin; this system returns 52.5% — a 30% relative gain in compositional separation, winning on 8/12 pairs (§4)
Training	None. Every model is used zero-shot, off-the-shelf

1. Problem framing: why this is hard

The five evaluation queries are deliberately chosen to break a naïve retriever. Decomposing them shows three distinct capabilities are required:

#	Query	Capability stressed	Why plain CLIP struggles
1	<i>bright yellow raincoat</i>	colour + garment	fine-grained fashion vocabulary is sparse in generic CLIP training
2	<i>business attire inside a modern office</i>	style + scene	needs scene grounding, not just garment
3	<i>blue shirt sitting on a park bench</i>	colour + garment + scene + pose	multi-constraint conjunction
4	<i>casual weekend outfit for a city walk</i>	style inference	no garment words at all — must infer hoodie/jeans/street from "vibe"
5	<i>a red tie and a white shirt</i>	compositional binding	the canonical CLIP failure

The actual mechanism of CLIP's failure (not just "it's bad at composition")

CLIP is trained with a contrastive objective over a **single pooled embedding per image**. That pooling is order- and position-agnostic: the image vector encodes *"there is red, there is white, there is a tie, there is a shirt"* but **not which attribute binds to which object**. Consequently **"red tie + white shirt"** and

"white tie + red shirt" map to nearly the same point in embedding space — they are literally the same bag of concepts.

This is an **architectural** limitation, not a capacity one. Scaling the encoder or fine-tuning does not remove it, because the information is destroyed at the pooling step. **Therefore the fix must reintroduce structure below the image level** — which is exactly what region decomposition does.

2. Approaches considered, and their trade-offs (*Deliverable 1*)

#	Approach	Strengths	Weaknesses	When it's the right call
A	Vanilla CLIP/OpenCLIP + FAISS	trivial, strong zero-shot, one model	bag-of-words : no colour ↔ garment binding; weak fine-grained fashion terms	a quick baseline, or when queries are single-concept
B	FashionCLIP + FAISS	big domain gain on garments/fabric/colour for ~zero cost	same architectural binding limitation	any fashion task — adopt as the <i>floor</i> , not the solution
C	Caption → LLM attribute extraction → hybrid	interpretable; good scene/style; handles query 4	error compounds (caption hallucination → wrong attribute); non-deterministic; extra GPU-heavy LLM	when captions are reliable and you need open-vocabulary attributes
D	CLIP zero-shot tagging vs controlled vocabulary	grounded in <i>pixels</i> not text; deterministic; cheap; vocabulary is editable	image-level only → still no binding; needs a curated vocabulary	structured, filterable attributes without training
E	Region / part decomposition	genuinely solves binding ; dataset-agnostic	extra segmentation cost at index time; fails on tiny/occluded garments	whenever compositionality is graded — as it is here
F	Object detector + scene graph + VQA per attribute, or a vector-DB service (Pinecone) + microservices	maximal expressiveness	large latency/complexity, more failure surface, no measured retrieval gain at this scale	very large production systems, not a 4-hour assignment

Decision. Adopt **B + D + E**, with a caption signal from **C** (BLIP only, *no* LLM extraction — grounding attributes in pixels beats grounding them in possibly-hallucinated text). Reject **F** as over-engineering: the assignment explicitly says to favour ML logic over indexing engineering, so I used FAISS + SQLite and spent the effort on ranking.

3. Chosen architecture (*Deliverable 2*)

3.1 Indexing pipeline (Part A) — `src/indexing/`

Per image, four artefacts are produced:

1. **FashionCLIP global embedding** → `global.faiss` — semantic/scene baseline.

2. **BLIP caption** → **MPNet sentence embedding** → `caption.faiss` — a *text-native* semantic view that captures vibe/context.
3. **CLIP zero-shot attributes** → SQLite — for each axis (environment / style / garment / colour) every vocabulary value is embedded through several **prompt templates and averaged** (prompt ensembling reduces sensitivity to phrasing), then the image is scored against that bank and softmax-normalised.
4. **SegFormer garment regions** → each garment is cropped, embedded by FashionCLIP, and independently tagged with its own **colour and fine-grained type** → `region.faiss` + SQLite.

Artefact (4) is the crux: it preserves *which colour sits on which garment*.

3.2 Retrieval pipeline (Part B) — `src/retrieval/`

Stage 1 — recall (sub-linear). The query is embedded by FashionCLIP (text tower) and MPNet; the union of top-N from `global.faiss` and `caption.faiss` forms a candidate pool (default 400).

Stage 2 — precision rerank. Only that small pool is scored with the expensive, feature-rich signals. This ANN-recall → rerank split is the standard web-scale pattern, and is why the logic is unchanged at 1M images (§7).

3.3 The ranking function — and why each term exists

$$\text{score} = w_1 \cdot \text{global_clip} + w_2 \cdot \text{caption_sim} + w_3 \cdot \text{attribute_match} + w_4 \cdot \text{region_binding}$$

(0.40)
(0.20)
(0.20)
(0.20)

Signal	What it fixes	Query it rescues
<code>global_clip</code>	overall semantics + scene	all
<code>caption_sim</code>	style/vibe inference — the caption "a man in a hoodie walking down a street" matches "casual weekend outfit for a city walk" with zero shared garment words	4
<code>attribute_match</code>	explicit, grounded multi-attribute filtering	1, 2
<code>region_binding</code>	compositional colour ↔ garment binding	5

Query parsing. A deterministic rule + synonym + adjacency parser converts the query into a `QuerySpec` : structured attributes plus `(colour, garment) bindings` ("a red tie and a white shirt" → `[(red, tie), (white, shirt)]`). I chose rules over an LLM because the schema is small and fixed, it is instant and reproducible, and it is trivially unit-testable — while remaining a drop-in point for an LLM parser later (§8).

Region-binding scoring. A binding scores high only if **some single region** matches *both* the garment type and the colour. Multiple bindings are combined as `0.5·mean + 0.5·min` : a query like "red tie AND white shirt" is a **conjunction**, so satisfying only half of it must not score like satisfying all of it (plain `mean` is too lenient); but plain `min` is brittle — one missed segmentation would zero an otherwise perfect image. The hybrid keeps partial credit while rewarding full satisfaction.

Two subtleties that materially affected quality (both found by measurement, not intuition):

1. **Per-signal normalisation is mandatory.** FashionCLIP cosines occupy a narrow band (~0.15–0.35) while attribute/region scores span [0, 1]. A raw weighted sum would silently drown the CLIP signal. Each signal is therefore **min-max normalised across the candidate pool** before weighting, so every term competes on equal footing. Signals that don't apply to a query (e.g. no binding present) are dropped and their weight **redistributed**.

2. **Bound attributes must be excluded from `attribute_match`**. That signal is *order-agnostic*: both `"red tie + white shirt"` and `"white tie + red shirt"` reduce to the colour set `{red, white}`. Counting those colours again pulls colour-swapped queries back **together**, diluting the order-sensitive `region_binding`. Excluding bound attributes improved measured compositional separation from **15.8%** → **23.7%** (§4).

4. Empirical results — *does it actually work?*

4.1 Compositionality: an objective, label-free measurement

Asserting "we beat CLIP" is cheap. I measure it. For a query **Q** and its **colour-swapped twin Q'** (identical words, colours exchanged), a bag-of-words retriever cannot tell them apart, so `top-k(Q)` and `top-k(Q')` are nearly the **same set**. A compositional retriever pulls **different** images. So **overlap@k is an inverse proxy for compositional sensitivity** — and it needs no human labels.

Both systems run over the **same index**; only the ranking differs. **12 pairs × k=10 = 120 slots** — deliberately sized so the mean is not dominated by single-image noise (at 5 pairs, one image moves the mean by 0.02, which is not a measurable difference).

Colour-swapped pair	vanilla CLIP	this system	
green top / yellow skirt	1.00 (<i>completely blind</i>)	0.10	✓
purple top / black trousers	0.70	0.20	✓
yellow top / blue skirt	0.80	0.30	✓
green jacket / white shirt	0.80	0.40	✓
brown coat / black trousers	0.90	0.50	✓
red skirt / white blouse	0.70	0.60	✓
blue shirt / red trousers	0.90	0.70	✓
black dress / white jacket	0.90	0.80	✓
black jacket / white trousers	0.70	0.70	=
pink blouse / black skirt	0.70	0.70	=
white blouse / blue jeans	0.40	0.50	✗
red tie / white shirt	0.50	0.80	✗
Mean overlap@10	0.750	0.525	−30.0%

Result: a 30% relative reduction in swap overlap, winning on 8/12 pairs (2 ties, 2 losses). The clearest case is *green top / yellow skirt*, where CLIP returns an **identical** top-10 for both phrasings (overlap 1.00) — the textbook failure — while this system separates them almost completely (0.10).

Where the gains concentrate — an informative pattern. The eight wins are exactly the pairs where vanilla CLIP is most blind (overlap 0.70–1.00), and the two losses are pairs CLIP *already* separated well (0.40, 0.50). In other words **the region-binding mechanism contributes most precisely where the baseline fails hardest**, and adds noise where the baseline was already correct. That asymmetry is the desired behaviour for

a corrective signal, and it suggests a natural refinement: make the binding weight *adaptive* to how confidently the dense signals already discriminate.

4.2 Honest negative result, diagnosed

The *red tie / white shirt* pair — the assignment's query 5 — **did not improve**. Rather than hide it, I traced the cause in data. The index originally held only **18 tie regions across 3,200 images**, because two filters removed small garments (an area threshold of 0.5% of image area, plus a cap of 6 regions/image applied to an *area-sorted* list, so a tie — the smallest item — was truncated away). I lowered the threshold to 0.15% and raised the cap, then re-indexed: regions rose **7,321 → 9,851** and ties **18 → 44**, but query 5 still did not improve.

The remaining ceiling is the corpus, not the algorithm. Fashionpedia is womenswear- and runway-heavy; 44 ties in 3,200 images (~1.4%), few of them red, means there is very little correct material to retrieve. The mechanism itself demonstrably works when the garments exist — *green top / yellow skirt* goes from 1.00 to 0.10. The fix for query 5 is therefore **dataset composition** (blend in menswear/formal imagery), not a change to the ranker.

This is the value of measurement: two plausible assumptions — "regions exist" and "more regions will fix it" — were both false, and only data revealed it.

4.3 Qualitative behaviour on the five official queries

Query	Score	Top result	Assessment
1	0.942	"a little girl in a yellow raincoat and red tights"	★ exact match
2	0.916	"a woman in a black suit and white shirt"	✅ business attire correct; office scene is the weaker axis (corpus holds only ~198 office-like images)
3	0.906	"a young girl sitting on a bench in the park"	✅ scene + pose nailed; colour binding softer
4	0.896	"a woman standing on the sidewalk in a white shirt and jeans"	★ correct casual/street inference from a query containing no garment words at all
5	0.702	"a man in a vest and red pants on the runway"	⚠️ weakest — corpus-limited (§4.2)

Latency: mean **318 ms** across the five queries (~150 ms warm; the first query pays a one-off cold-start cost).

4.4 Corpus composition (measured)

Zero-shot environment tagging over the 3,200 images confirms the required axes are present: urban street **765**, home interior **391**, park **292**, office **198**, with the remainder dominated by catalogue/runway "studio" imagery (1,177) — the known Fashionpedia bias, and the reason query 2's *scene* component is the weaker axis.

5. Shortcomings, and how I would address them

The assignment explicitly asks what the approach's weaknesses are. Honestly:

Shortcoming	Why it happens	How to fix
Small garments (ties, belts) under-detected	area threshold + region cap dropped them; measured only 18 ties / 3,200 images	lower threshold (done), or an accessory-specialised detector; weight regions by <i>saliency</i> not raw area
Multi-person images bind incorrectly	SegFormer parses clothing, not <i>people</i> — a red tie on person A and a white shirt on person B can satisfy both bindings	add a person detector and bind per person instance , requiring bindings to resolve within one individual
Attribute confidences are uncalibrated	softmax over a vocabulary gives <i>relative</i> , not probabilistic, scores; scales differ per axis	per-axis Platt scaling / temperature calibration on a small labelled set
Scene axis is corpus-limited	Fashionpedia is catalogue/runway-heavy; only ~6% office-like	blend in a scene-rich people dataset (COCO/Places) — the loader is dataset-agnostic by design
Rule-based parser misses negation/comparatives	"no tie", "more formal than" are out of grammar	swap in a small instruction-tuned LLM emitting the <i>same</i> QuerySpec JSON — the interface already exists
Weights are hand-set	no labelled validation set within the time budget	learn the fusion (§8.2)
Caption bias	BLIP describes salient subjects; may omit accessories	ensemble a second captioner, or condition captioning on detected regions

6. Modular code — logic separated from data (*Criterion 2*)

```

configs/      models.yaml · paths.yaml · retrieval.yaml  ← every knob; no constant in code
src/utils/    config loader, logger, typed schema (ImageRecord / RegionRecord / QuerySpec)
src/models/   thin wrappers: FashionCLIP · BLIP · MPNet · SegFormer · CrossEncoder
src/attributes/ vocabulary · zero-shot tagger · query parser
src/database/ SQLite metadata store + FAISS vector store
src/indexing/ PART A: dataset loader (dataset-agnostic) + indexer
src/retrieval/ PART B: HybridScorer (4 signals) + Retriever (2-stage)
src/evaluation/ official queries · contact sheet · compositionality swap test
tests/        pure-logic unit tests (parsing + compositional binding)
webapp/        optional Flask demo portal

```

Concretely: **business logic never imports HuggingFace directly** (only `src/models/` wrappers do), so swapping FashionCLIP → SigLIP is a one-line YAML change. **Vectors live in FAISS, metadata in SQLite** — never in filenames. Every stage passes typed dataclasses, so stages are independently testable, and the dataset loader only assumes “a folder of images” — Fashionpedia, DeepFashion, COCO or phone photos all work unchanged.

7. Scalability to 1M+ images (*Criterion 3*)

The retrieval **logic is size-invariant**; only the index type changes.

- **Recall stage:** `IndexFlatIP` → `IndexIVFFlat` or HNSW — a *one-line* change in `VectorStore.build(index_type="ivf")`. Ids and the `search()` contract are identical, so **nothing else moves**. Query cost goes from $O(N)$ to $\sim O(\sqrt{N})/O(\log N)$.
- **Rerank stage:** cost is **independent of corpus size** — only the fixed candidate pool (400) is scored with expensive signals. This is the key scalability property.
- **Memory:** $1M \times 512\text{-d fp16} \approx \mathbf{1\ GB}$ for the global index; regions $\sim 2.3\times$ that. With IVF-PQ compression (8–32 $\times$) the whole thing fits comfortably in RAM on one machine.
- **Metadata:** SQLite is fine into the low millions; the `MetadataDB` seam allows Postgres/ DuckDB. Crucially, structured attributes enable **SQL pre-filtering** (`environment=office AND EXISTS a tie region`) to shrink the ANN search space *before* it runs.
- **Indexing:** embarrassingly parallel — shard images across workers/GPUs and merge FAISS shards. Embeddings are computed once, offline. Measured: **3,200 images in ~ 11 min** on one 4 GB laptop GPU (~ 0.2 s/image) → ~ 57 GPU-hours for 1M on this hardware, or a few hours across a modest cluster.

8. Future work (*Deliverable 4*)

8.1 Adding locations (cities, places) and weather

Environment is already a **first-class structured axis**, so this is mostly a *data* change plus new scoring terms — no architectural rewrite:

1. **Vocabulary extension (no code path change).** Add `weather = [sunny, rainy, snowy, overcast...]` and richer `place` values to `src/attributes/vocab.py` with prompt templates. FashionCLIP tags them **zero-shot** immediately; the parser gains synonyms. This alone makes "*a raincoat on a rainy street*" prefer genuinely wet scenes.
2. **Specialised classifiers for precision.** Replace zero-shot tags with **Places365** for scene/venue and a dedicated sky/weather classifier — stored as two more attribute axes and two more (weighted) scoring terms.
3. **Geo metadata + reverse geocoding.** If images carry GPS EXIF, resolve to city/landmark and store it. This enables a powerful **hybrid**: a *hard SQL geo filter* ("in Paris") composed with *soft visual ranking* — exactly the pre-filter pattern from §7.
4. **Landmark recognition** for "in front of the Eiffel Tower" style queries — an embedding lookup against a landmark gallery, added as another region-like signal.
5. **Garment ↔ weather consistency prior.** A light term rewarding agreement between garment and weather (raincoat ↔ rain, parka ↔ snow) captures context the query *implies* but doesn't state — and would let the system answer "*dressed for bad weather*".

Because each is just "another axis + another weighted signal", they slot into the existing fusion with no change to the retrieval flow.

8.2 Improving precision

Ordered by expected return on effort:

1. **Build a labelled validation set** — the highest-value next step. Without relevance labels, tuning is guesswork; with a few hundred `(query, relevant_id)` pairs we get **precision@k / nDCG** and every change becomes measurable. The harness already accepts `--labels`.

2. **Learn the fusion instead of hand-setting weights.** With labels, fit logistic regression or **LambdaMART** over the four signals — replaces four hand-tuned constants with a trained ranker, and naturally learns *query-dependent* weighting.
3. **Contrastive fine-tuning on hard negatives.** LoRA-adapt FashionCLIP using **colour-swapped pairs** as explicit negatives, pushing binding into the *encoder* rather than only the scorer. This attacks the root cause identified in §1.
4. **Stronger colour grounding.** Ensemble the CLIP colour head with a classical **CIELAB** dominant-colour estimate per region (robust to lighting/white balance), which should directly lift queries 1, 3 and 5.
5. **Per-person binding** (see §5) — the biggest correctness fix for crowded scenes.
6. **Cross-encoder reranking**, already implemented behind a flag, enabled only after its uplift is *measured* rather than assumed.
7. **Query expansion** for style inference: expand "*weekend casual*" into likely garments via a learned style → garment prior, improving recall on vibe-only queries.

9. Zero-shot capability (*Criterion 4*)

Nothing in the system is trained on this corpus or on the queries. FashionCLIP, BLIP, MPNet and SegFormer are all used off-the-shelf; attributes come from **zero-shot CLIP classification** against an editable vocabulary. A new concept — a colour, garment, city or weather condition — is added by editing `vocab.py`, with **no retraining and no re-labelling** (only re-tagging). Query 4 is the practical demonstration: "*casual weekend outfit for a city walk*" contains **no garment label at all**, yet returns sidewalk/jeans imagery because the caption and attribute signals infer the vibe rather than matching a keyword.

Appendix A — Reproducing

```
pip install -r requirements.txt
python main.py index                                # Part A – build FAISS + SQLite
python main.py query "a red tie and a white shirt in a formal setting" -k 10 # Part B
python main.py evaluate                             # 5 official queries → contact sheet
python main.py compositional                        # the colour-swap test of §4.1
python webapp/app.py                               # optional visual portal
pytest -q                                           # unit tests (parsing + binding)
```

Verification of the technical requirements

- *Part A / Part B are distinct modules:* `src/indexing/` and `src/retrieval/`.
- "*Avoiding simple filename keyword matching*" — guaranteed structurally, not by promise: the dataset's filenames are MD5 hashes (`003d41dd20f271d27219fe7ee6de727d.jpg`) with zero semantic content, and `image_path` never enters any scoring computation — it appears in the ranking layer only as a field carried through for display. Ranking reads embeddings and structured attributes exclusively.
- *Context awareness:* "`a woman in a red dress in a park`" parses to `environment=park · upper_garment=dress · colors=red · binding(red→dress)` and returns "*a woman in a red dress sitting on a bench*" at 0.966.
- *Vector DB choice:* FAISS + SQLite — deliberately the simplest capable option, per the assignment's instruction to spend effort on ML logic rather than re-implementing storage.

Appendix B — Stack

FashionCLIP ([patrickjohncyh/fashion-clip](#)) · BLIP ([Salesforce/blip-image-captioning-base](#)) · MPNet ([all-mpnet-base-v2](#)) · SegFormer clothes parser ([mattdmjaga/segformer_b2_clothes](#)) · FAISS · SQLite. Hardware: single 4 GB NVIDIA RTX 2050.